Basics Day 6 — Session 6 (Hours 21–24)

Python Programming (Basic) • Data Structure Selection & Mini-Projects

Session 6 Overview

- Hour 21: Choosing the right structure — list vs set vs dict
- Hour 22: Data-structure drill circuit (guided practice)
- Hour 23: Mini-project: In-memory tracker
- Hour 24: Checkpoint 3: Data structures assessment

Hour 21: Choosing the Right Structure

Learning Outcomes

- Select list vs set vs dict based on task requirements
- Refactor an approach using a better-fitting structure
- Explain tradeoffs between structures in terms of readability and simplicity
- Apply a decision checklist: order, duplicates, key lookup, mutability

The Structure-Choice Problem

What We've Covered So Far

By the end of Session 5, you learned:

- **Lists** — ordered, mutable sequences
- **Tuples** — ordered, immutable sequences (fixed records)
- **Sets** — unordered collections of unique items
- **Dictionaries** — key-value mappings for fast lookup

Today's Shift

From knowing **what** each structure is → choosing **when** to use each structure

> 💡 Good programmers ask: "What structure makes this simpler, clearer, and safer?"

Decision Checklist

Five Key Questions

1. **Does order matter?** → List or Tuple
2. **Can duplicates happen?** → If unwanted, use Set
3. **Do I need to look something up by a key?** → Dictionary
4. **Is this a fixed record with a small number of parts?** → Tuple
5. **Will the collection change over time?** → List (mutable) or Set (mutable)

Comparing Structures: Quick Reference

Structure	Ordered?	Mutable?	Duplicates?	Best For
List	✅ Yes	✅ Yes	✅ Yes	Sequences, ordered collections
Tuple	✅ Yes	❌ No	✅ Yes	Fixed records (coordinates, dates)
Set	❌ No	✅ Yes	❌ No	Removing duplicates, membership tests
Dict	✅ Insertion order*	✅ Yes	❌ No (keys)	Lookup by name/ID, key-value pairs

Python 3.7+ maintains insertion order for dicts.

Example 1: Checking for Duplicates

The Task

You have a list of email addresses and need to identify if any are duplicates.

Approach 1: Using a List

```
emails = ["alice@example.com", "bob@example.com", "alice@example.com"]

# Check for duplicates by counting
for email in emails:
    if emails.count(email) > 1:
        print(f"{email} is duplicated!")
```

Problem: `count()` scans the entire list for each email — inefficient and harder to read.

Example 1: Better Approach with Set

Approach 2: Using a Set

```
emails = ["alice@example.com", "bob@example.com", "alice@example.com"]

# Convert to set to see unique items
unique_emails = set(emails)

if len(emails) != len(unique_emails):
    print("Duplicates found!")
    print(f"Original: {len(emails)}, Unique: {len(unique_emails)}")
```

Better because:

- Clearer intent: "I care about uniqueness"
- Simpler logic
- More efficient for large lists

Example 2: Looking Up Phone Numbers

The Task

Store contacts and look up phone numbers by name.

Approach 1: Using a List of Tuples

```
contacts = [
    ("Alice", "555-1234"),
    ("Bob", "555-5678"),
    ("Charlie", "555-9012")
]

# Lookup by name
name = "Bob"
for contact_name, phone in contacts:
    if contact_name == name:
        print(f"{name}: {phone}")
        break
```

Example 2: Better Approach with Dictionary

Approach 2: Using a Dictionary

```
contacts = {
    "Alice": "555-1234",
    "Bob": "555-5678",
    "Charlie": "555-9012"
}

# Direct lookup by name
name = "Bob"
if name in contacts:
    print(f"{name}: {contacts[name]}")
else:
    print(f"{name} not found")
```

Better because:

- Direct lookup by key (no loop needed)

Tradeoffs to Consider

Simplicity vs Power

- **Don't overengineer:** If a simple list works and is readable, use it
- **Don't underengineer:** If you're looping repeatedly to find items, consider a dict

Readability

- Choose the structure that makes your intent clearest to other programmers
- A dict says "I'm mapping keys to values"
- A set says "I care about uniqueness"

Performance (Basic-level awareness)

- Lists: Good for small collections and sequential access
- Sets: Fast membership testing (`item in set`)

Demo: Membership Lookup — List vs Set

Scenario

Check if a visitor is on a VIP list.

```
# Using a list
vip_list = ["Alice", "Bob", "Charlie", "Diana", "Eve"]

if "Charlie" in vip_list:
    print("VIP found in list!")

# Using a set
vip_set = {"Alice", "Bob", "Charlie", "Diana", "Eve"}

if "Charlie" in vip_set:
    print("VIP found in set!")
```

Key Insight

Both work! For small collections, either is fine. For hundreds or thousands of items

Lab: Refactor Challenge

Instructions (30 minutes)

Task: Given a small problem, first solve it using a list, then refactor to use a better structure (set or dict) and explain why the refactor improves the solution.

Problem: Check for duplicate names in a list of students.

```
students = ["Alice", "Bob", "Charlie", "Alice", "Diana", "Bob"]

# Step 1: Solve using a list (hint: use .count() or nested loops)

# Step 2: Refactor using a set

# Step 3: Write a comment explaining at least one tradeoff
```

Lab: Completion Criteria

You Must:

✓ Produce a working list-based solution ✓ Produce a working set-based refactor ✓ Write an explanation that includes at least one tradeoff (e.g., readability, simplicity, efficiency)

Example Explanation

> "The list approach uses `.count()` which works but scans the list repeatedly. The set approach converts the list to a set and compares lengths. This is clearer because it directly expresses the concept of 'uniqueness.' The tradeoff is that we lose the order information, but for this task, order doesn't matter."

Common Pitfalls — Hour 21

Pitfall 1: Overengineering

```
# Overkill: using a dict when a list is simplest
items = {"item1": "milk", "item2": "bread", "item3": "eggs"}

# Better: just use a list if you don't need key-based lookup
items = ["milk", "bread", "eggs"]
```

Pitfall 2: Choosing the wrong structure for lookup

```
# Inefficient: looping through a list to find an item repeatedly
# Better: use a dict if you need frequent lookups by key
```

Optional Extensions — Hour 21

Add a Feature

- Add a search or filter feature to your refactored solution
- Example: Find all students whose names start with a certain letter

Stay in Basics Scope

✓ Use basic loops and conditionals ✗ Don't use list comprehensions (Advanced topic)
✗ Don't use lambda functions or advanced filtering

Quick Check — Hour 21

Exit Ticket Question: If you need fast lookup by name, which structure fits best?

Model Answer: "A dictionary is the best fit because it allows direct lookup by key (the name) without needing to loop through all items. This makes the code clearer and more efficient."

Recap:

- Use lists for ordered sequences
- Use sets for removing duplicates or membership testing
- Use dicts for key-value lookups
- Choose the structure that makes your intent clearest

Hour 22: Data-Structure Drill Circuit

Learning Outcomes

- Practice with lists, tuples, sets, and dicts in short timed tasks
- Apply the right structure to solve focused problems
- Complete coding tasks under time constraints
- Explain solutions verbally to reinforce understanding

The Circuit Format

What Is a Drill Circuit?

A rotation through 4 coding stations — 12 minutes per station:

1. **Station 1:** Filter numbers > 10 (list)
2. **Station 2:** Unique emails (set)
3. **Station 3:** Word frequency (dict)
4. **Station 4:** Coordinates unpacking (tuple)

Goal

Finish at least 3 of 4 stations with working code. At the end, share one solution verbally with the class.

Circuit Goals and Mindset

Finish Before Optimizing

- Get a working solution first
- Then improve if time allows
- Don't get stuck perfecting Station 1 — move on!

Test with Small Examples

- Run your code with 2-3 test inputs
- Make sure it produces the expected output
- Fix bugs before moving to the next station

No Collaboration During Stations

Work independently during the 12-minute rotations. Share and discuss during the 5-

Demo: Station 1 Example (Quick Walkthrough)

Task: Filter Numbers > 10

Given a list of numbers, create a new list containing only numbers greater than 10.

```
numbers = [5, 12, 3, 18, 7, 21, 9, 15]

# Create a new list with only numbers > 10
filtered = []
for num in numbers:
    if num > 10:
        filtered.append(num)

print(filtered)   # [12, 18, 21, 15]
```

Expected Output

```
[12, 18, 21, 15]
```

Station 1: Filter Numbers > 10 (List)

Task (12 minutes)

Given this list of numbers, create a new list containing only numbers greater than 10.

```
numbers = [5, 12, 3, 18, 7, 21, 9, 15, 2, 25, 8, 30]

# Your code here:
# 1. Create an empty list
# 2. Loop through numbers
# 3. If number > 10, add it to the new list
# 4. Print the result
```

Expected Output: `[12, 18, 21, 15, 25, 30]`

Station 2: Unique Emails (Set)

Task (12 minutes)

Given a list of email addresses (with duplicates), print only the unique emails.

```
emails = [
    "alice@example.com",
    "bob@example.com",
    "alice@example.com",
    "charlie@example.com",
    "bob@example.com",
    "diana@example.com"
]

# Your code here:
# 1. Convert the list to a set
# 2. Print the unique emails (order doesn't matter)
```

Expected Output: A set or list with 4 unique emails (order may vary).

Station 3: Word Frequency (Dict)

Task (12 minutes)

Given a list of words, count how many times each word appears.

```
words = ["apple", "banana", "apple", "orange", "banana", "apple"]

# Your code here:
# 1. Create an empty dictionary
# 2. Loop through words
# 3. If word is in dict, increment count; otherwise, set count to 1
# 4. Print the dictionary
```

Expected Output: `{'apple': 3, 'banana': 2, 'orange': 1}`

Station 4: Coordinates Unpacking (Tuple)

Task (12 minutes)

Given a list of (x, y) coordinate tuples, print each coordinate and calculate the total distance from the origin (0, 0).

```
coordinates = [(2, 3), (5, 1), (3, 4), (1, 2)]

# Your code here:
# 1. Loop through coordinates
# 2. Unpack each tuple into x and y
# 3. Print x and y
# 4. Calculate distance: sqrt(x*x + y*y) — use math.sqrt
# 5. Sum all distances and print total

import math
```

Expected Output: Print each coordinate and a total distance sum.

Circuit Debrief (5 minutes after stations)

Share-Out Questions

1. Which station was easiest? Which was hardest?
2. Which data structure felt most natural for its task?
3. Did anyone find a different approach? Share it!

One Solution Verbally

Each learner (or a few volunteers) explains one station solution to the class in 1-2 minutes.

Lab: Completion Criteria — Hour 22

You Must:

✓ Complete at least 3 of 4 stations with working code ✓ Produce correct output for the stations you completed ✓ Explain one solution verbally during the debrief

Success Markers

- Code runs without errors
- Output matches expected format
- Can articulate why you chose that structure

Common Pitfalls — Hour 22

Pitfall 1: Rushing and Creating Syntax Errors

```
# Example: Forgetting colons or indentation
for num in numbers
    if num > 10   # Missing colon
```

Fix: Slow down, test often, read error messages carefully.

Pitfall 2: Not Testing with Small Examples

Don't wait until the end to run your code — test after each step!

Optional Extensions — Hour 22

Add an Extra Station (if time allows)

Station 5: Nested list table formatting Given a list of lists representing rows, print each row as a formatted table.

```
table = [
    ["Name", "Age", "City"],
    ["Alice", "25", "NYC"],
    ["Bob", "30", "LA"]
]

# Format and print as a table
```

Stay in Basics Scope: Use loops and string formatting — no pandas or external libraries.

Quick Check — Hour 22

Exit Ticket Question: Name one thing you'll use dicts for in real work.

Example Answers:

- "Mapping user IDs to user profiles"
- "Counting occurrences of items (like word frequency)"
- "Storing configuration settings as key-value pairs"

Recap:

- Practiced lists, tuples, sets, and dicts in focused tasks
- Completed at least 3 of 4 stations
- Explained solutions verbally to solidify understanding

Hour 23: Mini-Project — In-Memory Tracker

Learning Outcomes

- Combine data structures into a coherent mini application
- Practice clean output and user-friendly updates
- Choose between list of dicts or dict of dicts for storage
- Build a menu-driven program with add, list, and search features

Mini-Project Overview

What We're Building

A small in-memory tracker application with:

- **Add:** Add a new record
- **List:** Display all records
- **Search:** Find a record by a key (name, title, etc.)

Storage Model Options

- **List of dicts:** `[{"name": "Alice", "phone": "555-1234"}, {...}]`
- **Dict of dicts:** `{"Alice": {"phone": "555-1234", "email": "..."}, ...}`

No File I/O Yet

All data is stored in memory (variables) — it disappears when the program ends. We'll

Project Choices

Pick One (or propose your own):

A) Expense Tracker

- Add expenses: `{date, amount, category}`
- List all expenses
- Search by category

B) Contact List

- Add contacts: `{name, phone, email}`
- List all contacts
- Search by name

C) Book Library

Requirements: Minimum Features

Must Include:

1. **Add** — add a new record to the collection
2. **List** — display all records in a readable format
3. **Search** — find and display a specific record

Must Use:

- At least one **dictionary** to store key-value pairs
- At least one **list** to hold multiple records (if using list of dicts)

Optional (but recommended):

- A menu loop that lets the user choose actions
- Input validation (e.g., don't add empty names)

Demo: Scaffolding a Menu and Add Feature

Step 1: Menu Structure

```
def main():
    contacts = []  # Storage: list of dicts

    while True:
        print("\n--- Contact Tracker ---")
        print("1. Add Contact")
        print("2. List Contacts")
        print("3. Search Contact")
        print("4. Quit")

        choice = input("Choose an option: ")

        if choice == "1":
            add_contact(contacts)
        elif choice == "2":
            list_contacts(contacts)
        elif choice == "3":
            search_contact(contacts)
        elif choice == "4":
            print("Goodbye!")
            break
```

Demo: Add Feature

Step 2: Add Function

```
def add_contact(contacts):
    name = input("Enter name: ")
    phone = input("Enter phone: ")
    email = input("Enter email: ")

    contact = {"name": name, "phone": phone, "email": email}
    contacts.append(contact)

    print(f"Added contact: {name}")
```

Key Points

- Collect input from user
- Store in a dictionary
- Append to the list

Lab: Mini-Project (30 minutes)

Instructions

Choose one of the three project options (Expense Tracker, Contact List, or Book Library) and implement:

1. **Add feature** — collect input and store in your data structure
2. **List feature** — print all records in a readable format
3. **Search feature** — find a record by name/title/category

Starter Template

```
def main():
    # Choose your storage model:
    records = []  # List of dicts, OR
    # records = {}  # Dict of dicts

    while True:
        # Menu here
```

Lab: Completion Criteria — Hour 23

You Must:

✓ Implement all three minimum features: add, list, search ✓ Use at least one dict and one list meaningfully ✓ Produce clear, readable output (not raw dict printing) ✓ Handle basic cases (e.g., search returns "not found" if missing)

Example Good Output

```
--- Contact List ---
1. Alice - 555-1234
2. Bob - 555-5678
```

Example Bad Output

```
[{'name': 'Alice', 'phone': '555-1234'}, {'name': 'Bob', 'phone': '555-5678'}]
```

Common Pitfalls — Hour 23

Pitfall 1: Messy Global Variables

```
# Bad: using global variables everywhere
contacts = []

def add_contact():
    global contacts
    # ...
```

Better: Pass the data structure as a parameter to functions.

Pitfall 2: Hard-to-Read Printing

```
# Bad: printing raw dictionaries
print(contacts)

# Better: format nicely
for contact in contacts:
```

Optional Extensions — Hour 23

Add More Features (Stay in Basics Scope)

- **Update:** Modify an existing record
- **Delete:** Remove a record by name
- **Sort:** Display records in alphabetical order

Example: Sort Contacts

```
def list_contacts(contacts):
    # Advanced topic preview: sorting a list of dicts.
    # For basics, you might just display unsorted or use simpler lists.
    sorted_contacts = sorted(contacts, key=lambda c: c['name'])
    for contact in sorted_contacts:
        print(f"{contact['name']} - {contact['phone']}")
```

> ⚠️ **Note:** Using `lambda` here is an Advanced topic — for Basics, you can use a

Quick Check — Hour 23

Exit Ticket Question: What data structure did you choose and why?

Example Answers:

- "I used a list of dicts because each contact is a record with multiple fields, and I want to keep them in order."
- "I used a dict of dicts because I want fast lookup by name as the key."

Recap:

- Built a mini in-memory tracker app
- Combined dicts and lists to store structured data
- Practiced clean output and basic menu-driven programs

Hour 24: Checkpoint 3 — Data Structures Assessment

Learning Outcomes

- Demonstrate confident use of core data structures and iteration
- Build a working program that meets a rubric
- Handle missing keys safely (no KeyError)
- Produce clear, formatted output

Checkpoint 3 Overview

What Is Checkpoint 3?

A timed assessment (45-60 minutes) where you build a simple application from scratch to demonstrate mastery of:

- Lists, tuples, sets, and dictionaries
- Iteration with `for` loops
- Safe key access (`dict.get()` or `if key in dict`)
- Clear output formatting

Optional Quiz (10 minutes)

After the lab, you may take a short multiple-choice quiz on data structures.

Assessment Task: Simple Contacts (In-Memory)

Requirements

Build a contacts program that:

1. Stores contacts as name → phone in a **dictionary**
2. Adds at least **5 contacts** (hardcoded or via input)
3. **Search by name** — print phone if found, message if not found
4. **List all contacts** — print all names and phones (sorted optional)

No File I/O

All data is in memory (variables) — no reading/writing files.

Rubric and Success Criteria

You Must:

✓ **Search works** for both existing and missing names ✓ **Clear output** — not raw dict printing ✓ **No KeyError** — handle missing keys gracefully ✓ Code runs without syntax errors

Example Success

```
Search for: Alice
Alice: 555-1234

Search for: Zoe
Zoe not found.

All Contacts:
  Alice: 555-1234
  Bob: 555-5678
  Charlie: 555-9012
```

Demo: How to Test with 2-3 Example Inputs

Step 1: Add Test Contacts

```
contacts = {
    "Alice": "555-1234",
    "Bob": "555-5678",
    "Charlie": "555-9012"
}
```

Step 2: Test Search Function

```
def search_contact(contacts, name):
    if name in contacts:
        print(f"{name}: {contacts[name]}")
    else:
        print(f"{name} not found.")

# Test with existing name
search_contact(contacts, "Alice")   # Should print: Alice: 555-1234
```

Demo: Safe Key Access

Method 1: Check with `in`

```
name = "Alice"
if name in contacts:
    print(contacts[name])
else:
    print("Not found")
```

Method 2: Use `dict.get()`

```
name = "Alice"
phone = contacts.get(name, "Not found")
print(phone)
```

Both methods avoid `KeyError` when the key doesn't exist.

Lab: Checkpoint 3 (45-60 minutes)

Task

Build the Simple Contacts program following the rubric:

```
# Step 1: Create a dictionary with at least 5 contacts
contacts = {
    "Alice": "555-1234",
    # Add 4 more...
}

# Step 2: Implement search function
def search_contact(contacts, name):
    # Your code here
    pass

# Step 3: Implement list function
def list_contacts(contacts):
    # Your code here
    pass

# Step 4: Test with 2-3 searches
search_contact(contacts, "Alice")
```

Lab: Completion Criteria — Hour 24

Required:

✓ Search works for existing names ✓ Search handles missing names gracefully (no KeyError) ✓ List displays all contacts with clear formatting ✓ At least 5 contacts stored in the dictionary

Bonus (Optional):

✓ Sorted output (alphabetically by name) ✓ Case-insensitive search (e.g., "alice" finds "Alice")

Common Pitfalls — Hour 24

Pitfall 1: Forgetting to Normalize Case

```
# Problem: "alice" doesn't match "Alice"
if name in contacts:
    # Won't find if case differs
```

Fix: Normalize to lowercase for searching:

```
name = name.lower()
# Store all keys in lowercase, or use .get() with case handling
```

Pitfall 2: Printing Dict Directly

```
# Bad: raw dict output
print(contacts)

# Better: formatted loop
```

Optional Extensions — Hour 24

Allow Partial Match Search

Search for contacts where the search term is a substring of the name:

```
def search_partial(contacts, keyword):
    results = []
    lowered_keyword = keyword.lower()
    for name in contacts:
        if lowered_keyword in name.lower():
            results.append(name)

    if results:
        for name in results:
            print(f"{name}: {contacts[name]}")
    else:
        print("No matches found.")
```

Stay in Basics Scope: This version stays in Basics scope by using a simple loop

Optional Quiz (10 minutes)

Sample Questions

Q1: What's the difference between `dict.get(key)` and `dict[key]`?

A: `dict.get(key)` returns `None` (or a default) if the key doesn't exist; `dict[key]` raises `KeyError` if the key is missing.

Q2: Which structure removes duplicates automatically?

A: Set

Q3: Which structure is best for ordered, fixed records like coordinates?

A: Tuple

Quick Check — Hour 24

Exit Ticket Question: What's the difference between `dict.get()` and `dict[key]`?

Model Answer: "`dict[key]` will raise a KeyError if the key doesn't exist. `dict.get(key)` returns `None` by default (or a custom default value) if the key is missing, making it safer for handling optional keys."

Recap:

- Completed Checkpoint 3: Simple Contacts program
- Demonstrated safe key access and clear output
- Practiced data structure selection and iteration

Scope Guardrail Reminder

Stay in Basics Scope

✓ Core data structures: list, tuple, set, dict ✓ Basic iteration with `for` and `while` ✓ Simple functions and imports ✓ Basic input/output and string formatting

Not Yet (Advanced Topics)

✗ Named tuples or collections module ✗ Performance analysis / Big O notation ✗ Advanced data modeling or custom classes ✗ List comprehensions (introduced later) ✗ Lambda functions or decorators

Session 6 Wrap-Up

What We Covered Today

- **Hour 21:** Choosing the right structure based on task requirements
- **Hour 22:** Drill circuit with lists, tuples, sets, and dicts
- **Hour 23:** Mini-project combining data structures into an app
- **Hour 24:** Checkpoint 3 assessment with Simple Contacts

Key Takeaways

- Decision checklist: order, duplicates, key lookup, mutability
- Practice refactoring to better-fitting structures
- Build mini-projects to integrate multiple structures
- Handle edge cases safely (no KeyError)

Homework and Practice

Practice Tasks

1. **Refactor Exercise:** Take a program you wrote earlier and identify one place where a different data structure would be clearer.
2. **Extend Mini-Project:** Add an update or delete feature to your tracker app.
3. **Word Counter:** Write a program that counts word frequency in a sentence using a dictionary.

Preparation for Session 7

- Review conditionals (`if/elif/else`) from earlier sessions
- Think about how to validate user input
- Practice writing functions that return values

Next Session Preview: Session 7 (Hours 25–28)

Topics Coming Up

- **Hour 25:** Conditionals — `if/elif/else` and boundaries
- **Hour 26:** Functions — defining, calling, returning values
- **Hour 27:** Function parameters and scope
- **Hour 28:** Checkpoint 4 — Functions + data structures

Get Ready To:

- Write reusable functions
- Structure programs with clear logic flow
- Combine functions with data structures

Questions?

Session 6 Complete!

You've mastered:

- Choosing the right data structure for the job
- Refactoring to improve clarity and simplicity
- Building mini-projects with multiple structures
- Demonstrating proficiency in Checkpoint 3

Keep practicing — see you in Session 7!